

Informe técnico de la aplicación CAT-IA

Autoras: Antonella Cuvertino y Miriam Hammami.

Resumen de la solución

CAT-IA es una aplicación web local por capas que transforma un ticket en un borrador de atención verificable. FastAPI expone la API y sirve un frontend HTML/CSS/JavaScript. Python implementa ingesta, recuperación, orquestación, contratos y evaluación. Se prioriza código pequeño que el estudiante pueda explicar sobre una arquitectura de microservicios que añadiría despliegue y comunicación innecesarios para nueve documentos.

Requisitos funcionales y aceptación

- RF01. Validar tickets de 12 a 4000 caracteres, impacto y parámetros. Entradas inválidas reciben HTTP 422.
- RF02. Recuperar documentos internos y externos, con k configurable entre 1 y 8 y filtro de tipo. Ninguna fuente externa necesita red durante la consulta lexical.
- RF03. Construir contexto limitado y prompts versionados. El ticket no se concatena dentro del mensaje de sistema.
- RF04. Clasificar en Soporte Técnico, Facturación, Dudas Comerciales, Sugerencias o No determinado; validar el JSON con Pydantic.
- RF05. Determinar prioridad mediante impacto declarado. Seguridad o caída organizacional: Alta/1 hora hábil; interrupción individual o afectación colectiva: Media/4; resto: Baja/24.
- RF06. Comprobar existencia del identificador y pertenencia exacta del extracto al contexto. Una salida inválida produce abstención.
- RF07. Mostrar fuentes, puntajes, tiempos, versión del prompt y recorrido. Descargar un resultado JSON con intervención explícita del usuario.
- RF08. Registrar trazas técnicas sin texto del ticket ni borrador. Cada solicitud tiene un UUID; no existe conversación compartida entre usuarios.
- RF09. Evaluar recuperación y clasificación contra etiquetas. Dejar fidelidad y relevancia pendientes si no se realizó evaluación semántica.

Requisitos no funcionales

RNF01: ejecución reproducible en Python 3.12 y dependencias fijadas. RNF02: interfaz adaptable a móvil, etiquetas asociadas a controles y navegación por teclado. RNF03: acceso por loopback, comprobación de origen para POST y renderizado de texto con `textContent` para evitar interpretar HTML del modelo.

RNF04: corpus y prompts identificables por SHA-256. RNF05: límite de entrada, k, contexto y salida para acotar recursos. RNF06: tiempo máximo HTTP de inferencia de 120 segundos; la latencia real se mide por equipo, no se promete un SLA técnico universal.

Arquitectura

El diagrama editable está en `docs/arquitectura.mmd` y su representación en `deliverables/arquitectura.pdf`. Hay dos recorridos: ingesta al arranque y consulta por ticket. En la ingesta, el manifiesto controla documentos activos, rutas y hashes; luego se fragmentan y vectorizan. En la consulta, el agente prepara el ticket, recupera evidencia, aplica política, invoca al generador, valida y presenta el borrador al operador.

Capas: presentación (`web/`), transporte (`catia/api.py`), aplicación (`catia/agent.py`), conocimiento (`catia/retrieval.py`), generación (`catia/generation.py`), contratos (`catia/schemas.py`) y configuración (`config/` y `prompts/`). Las dependencias apuntan desde la API al agente y desde este a retriever y generador; las pruebas pueden reemplazar el proveedor sin modificar el dominio.

Datos y trazabilidad

`sources.json` identifica nueve fuentes. Internas: INT-SLA, INT-ACC, INT-FAC, INT-COM, INT-SUG, INT-NET e INT-SEC. Externas: EXT-COOKIE y EXT-NET. Los documentos internos son simulados. Las fuentes externas son síntesis editoriales basadas en páginas oficiales consultadas; el texto original puede cambiar y los nombres de controles dependen de la versión del navegador. El sistema no afirma tener una copia íntegra, búsqueda web en vivo ni actualización automática.

Cada fragmento conserva `doc_id`, `id`, `texto`, `título`, `origen`, `tipo`, `versión`, `hash` del documento y `offsets` por palabra. El hash detecta cambio de bytes, no certifica veracidad ni firma autoría. Si el contenido cambia sin actualizar el manifiesto, la ingesta falla para forzar una revisión editorial. Para aceptar cambios: revisar la fuente, actualizar versión y fecha, ejecutar `scripts.source_hashes` y reiniciar. No actualizar hashes automáticamente para ocultar cambios no revisados.

Chunking y recuperación

Se usan ventanas de 110 palabras con solapamiento de 20; el avance es 90. Los documentos son cortos y se evita dividir por caracteres a mitad de una palabra. El solapamiento reduce pérdida de contexto de una frontera, a costa de redundancia. Los `offsets` permiten reconstruir la procedencia; una mejora sería segmentar por secciones cuando el corpus crezca.

La ruta por defecto vectoriza título y contenido con TF-IDF de unigramas y bigramas, normaliza acentos y filtra una lista básica de palabras funcionales. La similitud coseno determina el ranking. TF-IDF es vectorial disperso y léxico: no debe presentarse como un modelo neuronal de embeddings. No resuelve bien sinónimos ausentes y paráfrasis lejanas. No se instala una base vectorial externa porque la matriz completa cabe en memoria; para miles de documentos se reevaluaría persistencia e índices aproximados.

La ruta hybrid solicita vectores densos reales mediante Ollama /api/embed, normaliza su norma y combina 0,55 de coseno denso con 0,45 de coseno TF-IDF. Esto es una suma ponderada, no un reranker ni RRF. El peso es inicial y requiere experimento. Los puntajes no son probabilidades de acierto. El umbral lexical 0,06 no está calibrado automáticamente para modo híbrido; debe ajustarse con desarrollo y casos fuera de dominio, pues similitudes densas positivas pueden reducir abstención.

Después del filtro se conserva hasta k=4 y hasta 6500 caracteres de texto. Se omiten fragmentos completos que no caben. Este presupuesto limita el texto documental, no cuenta exactamente todos los tokens: la envoltura JSON, metadatos y ticket también ocupan contexto. Ollama tiene num_ctx=4096 en la configuración propuesta; se debe medir tokenización y truncamiento antes de expandir el corpus.

Prompt engineering

La versión activa es v3. Después de evaluar v2 se observaron errores al distinguir cotizaciones de incidencias y phishing de otras categorías. v3 explicita categorías por intención, separa compras futuras de cargos existentes e incluye incidentes de seguridad en Soporte Técnico. El historial y los errores están en reports/OPTIMIZACION_PROMPT.md; v1 y v2 se conservan para comparación. Repetir el test observado es regresión, no evaluación independiente.

v1 es una base zero-shot con tarea, categorías y JSON. v2 añade rol, objetivo, restricciones, prioridad de fuentes, manejo de falta de evidencia, contrato, justificación breve y tres ejemplos de criterio. Los ejemplos comerciales, de facturación y ambigüedad ayudan a distinguir conceptos sin usar tickets de prueba como ejemplos. No se pide exponer razonamiento interno; se pide una justificación breve que pueda contrastarse con citas.

build_messages guarda instrucciones en system y serializa ticket/documents como datos en user. Esto mejora separación, pero no hace al modelo inmune a prompt injection. El validador limita citas inexistentes y esquemas inválidos; una cita verdadera puede acompañar una recomendación falsa, por lo que sigue siendo necesaria revisión semántica. La prioridad interna tiene precedencia para reglas organizacionales; las fuentes externas complementan soporte técnico. El prompt establece esta jerarquía, pero no hay un detector semántico automático de conflictos implementado.

Generación y control

El esquema de evidencias se construye por solicitud: solo ofrece los ids y oraciones presentes en los fragmentos recuperados. El LLM selecciona entre esos pares mediante un JSON Schema dinámico. La validación posterior vuelve a comprobar pertenencia; limitar opciones no prueba que la oración elegida respalde la recomendación. Se redujeron metadatos enviados al LLM para evitar confundir doc_id con id de fragmento; la respuesta de la API conserva la trazabilidad completa.

OllamaGenerator invoca /api/chat con stream=false, JSON Schema, temperatura 0, num_predict=1100 y timeout de 120 segundos. Pydantic verifica campos, enumeraciones, tamaños y propiedades adicionales. Temperatura 0 reduce variación; no garantiza determinismo ni verdad. El modelo propuesto es qwen2.5:1.5b por tamaño moderado para práctica local; elección sujeta a rendimiento y revisión de licencia/model card por el equipo antes de otro uso.

DemoGenerator usa palabras clave y extractos. Es un doble funcional para ensayar interfaz y probar integraciones locales. Nunca sustituye silenciosamente un fallo de Ollama: provider_error indica fallo real, invalid_output indica salida no validada e insufficient_context indica ausencia de evidencia recuperada. El

estado draft solo significa que pasó el contrato y las citas, no que un humano aprobó la respuesta.

La política de prioridad es una función determinista y se referencia a INT-SLA. Su modificación exige cambiar tanto política documental como función y pruebas: no es una regla aprendida ni inferida del texto por el LLM. La app muestra horas hábiles de primera respuesta, sin convertirlas en fecha ni considerar feriados.

API

GET / sirve la interfaz. GET /api/health muestra modo, modelo, configuración y tamaño del corpus; no comprueba por sí solo que Ollama pueda inferir. GET /api/sources devuelve el manifiesto. POST /api/tickets/analyze recibe text, impact, service_down, security_incident, top_k opcional y source_filter. /docs expone Swagger/OpenAPI y requiere internet para sus recursos visuales; /openapi.json no lo requiere.

Ejemplo de entrada: {"text":"Mi factura tiene un cobro duplicado","impact":"individual","service_down":false,"security_incident":false,"top_k":4,"source_filter":"all"}.

La respuesta incluye decision, priority, first_response_hours, sources, policy, steps, timings, usage, trace_id, model, mode, prompt_version y hashes. Las salidas degradadas siguen siendo una respuesta de negocio HTTP 200 con status explícito. Una falla en recuperación devuelve 503; entrada inválida 422; origen ajeno 403.

Seguridad y privacidad

La redacción básica reemplaza correos, RUT con formato conocido y secretos etiquetados como contraseña/token. No detecta todas las formas de datos personales, teléfonos o secretos sin etiqueta. Usar solo datos ficticios en la defensa. El backend registra categoría y metadatos técnicos, no el texto; la descarga JSON sí contiene un resumen y debe tratarse con cuidado. No se incluye autenticación ni autorización multiusuario: mantener bind en 127.0.0.1 y no exponer públicamente.

Decisiones y alternativas

ADR01: monolito modular frente a microservicios. Reduce operación y facilita seguir el flujo durante la defensa; la separación por módulos conserva mantenibilidad.

ADR02: RAG frente a fine-tuning. Las políticas cambian y necesitan procedencia; actualizar documentos es más directo que entrenar pesos y no requiere un dataset de entrenamiento grande.

ADR03: TF-IDF como baseline y embeddings opcionales. Garantiza reproducción sin descargar un modelo, pero se reconoce la menor cobertura semántica. Se compara antes de aumentar complejidad.

ADR04: Pydantic y citas exactas frente a aceptar texto libre. Permite manejar fallos de contrato y referencias inventadas; no prueba implicación semántica.

ADR05: política determinista y borradores humanos. Evita delegar compromisos contractuales simples al generador y mantiene el control del operador.

ADR06: trazas locales frente a LangSmith obligatorio. Evita credenciales y exportación del ticket; conserva ids, corpus, prompt, tiempos y uso. Se inspira en trazabilidad del curso, pero no afirma integrar

LangSmith. Una futura integración requeriría decidir qué datos exportar.

Referencias técnicas

Material docente aportado de RAG, evaluación y LangSmith. Ollama: <https://docs.ollama.com/api/chat>, <https://docs.ollama.com/api/embed> y <https://docs.ollama.com/capabilities/structured-outputs>. Fuentes externas de dominio detalladas en `data/sources.json`. GitHub informa el retiro de GitHub Models: <https://docs.github.com/en/github-models>; por eso no se copia el endpoint histórico de la propuesta original.